What if your secrets are stored in a dedicated backend like HashiCorp Vault or Azure Key Vault? In that case, you can use a project like *External Secrets* (https://github.com/external-secrets/kubernetes-external-secrets). As you can guess from its name, this project lets you generate a Secret from an external source. The `ExternalSecret` object would be safe to store in your repository and put under version control. When the `ExternalSecret` manifest is applied to a Kubernetes cluster, the External Secrets controller fetches the value from the configured external source and generates a standard Secret object that can be used within a Pod.

NOTE If you're interested in learning more about how to secure Kubernetes Secrets, you can check out chapter 7 of *GitOps and Kubernetes* by Billy Yuen, Alexander Matyushentsev, Todd Ekenstam, and Jesse Suen (Manning, 2021) and *Kubernetes Secrets Management* by Alex Soto Bueno and Andrew Block (Manning, 2022). I won't provide more information here, since this is usually a task for the platform team, not developers.

When we start using ConfigMaps and Secrets, we must decide which policy to use to update configuration data and how to make applications use the new values. That's the topic of the next section.

14.2.3 Refreshing configuration at runtime with Spring Cloud Kubernetes

When using an external configuration service, you'll probably want a mechanism to reload the applications when configuration changes. For example, when using Spring Cloud Config, we can implement such a mechanism with Spring Cloud Bus.

In Kubernetes, we need a different approach. When you update a ConfigMap or a Secret, Kubernetes takes care of providing containers with the new versions when they're mounted as volumes. If you use environment